

Inventory, Idempotency & ACK

Commit business work first • Survive redelivery • Protect concurrent stock updates

SAY IT IN THE INTERVIEW

“Kafka can redeliver a record after the database has committed. I use `eventId` to detect an already processed event. Stock changes, the processed-event record and the result outbox are in one transaction. The listener acknowledges only after that transaction succeeds, including the duplicate-skip path.”

01 ONE TRANSACTION, THREE RELATED WRITES

- **Idempotency:** processing the same event again must not repeat its business effect. Our key is **`eventId`**, not the Kafka offset or `orderId`.
- For a new event: load inventory → save `ProcessedEvent` → reserve stock or record rejection → save result outbox. A technical exception rolls back this transaction.
- **`processed_events.event_id`** has a unique constraint. The existence check helps normal duplicates; the database constraint protects concurrent duplicate inserts.

02 ACK BELONGS AFTER THE TRANSACTION

- Consumer calls the separate **`@Transactional`** service. The Spring proxy commits before control returns to the listener; then `acknowledge()` runs.
- **Manual ACK** tells the listener container that processing is complete so it can commit the consumed position. It does not delete the Kafka record.
- Crash after DB commit but before offset commit? Redelivery finds `eventId`, skips business work, and reaches ACK. ACK before DB commit could lose work after a crash.
- Technical failure escapes the service, so the normal ACK line is not reached. Retry/DLT behavior is covered in Chapter 05.

03 TWO DIFFERENT STOCK CONCERNS

- **Insufficient stock:** a valid business outcome. Commit the rejection result + processed marker; acknowledge it. It is not a poison event.
- **Optimistic locking:** `@Version` detects another transaction changing the same inventory row before our update commits. A stale update fails instead of silently overwriting.
- Stock is changed on a managed JPA entity; dirty checking persists it on commit. Kafka is keyed by **`orderId`**, so different orders for one product can still contend.

04 FOLLOW-UP: IS THIS EXACTLY-ONCE?

- “I use at-least-once delivery with idempotent database processing. I do not claim Kafka exactly-once semantics or one distributed Kafka/MySQL transaction.”
- `eventId` must stay stable on replay. A new `eventId` looks like a new business event; retention of processed IDs must consider possible late redelivery.

YAAD RAKHO: same `eventId` → skip effect; successful DB commit → ACK.